JavaScript Basics - Assignment 10

Asynchronous JavaScript | Scenario-Based Practice

Topics Covered: Synchronous vs Asynchronous JavaScript, Callbacks, Callback Hell, Promises, resolve/reject, then/catch, Promise Chaining, async/await, Error Handling

Important Instructions

- Read each scenario carefully and plan the sequence of steps before writing code.
- Create a separate JavaScript file for each question. Add an HTML file for each question when needed.
- Use meaningful function and variable names and keep the code properly indented.
- Use only the concepts taught in class. Do not use frameworks or external libraries.
- Use console.log() unless another output method is requested.
- Test both success and failure cases whenever error handling is required.

Submission Instructions

- Submit through a link to your GitHub repository.
- Keep a separate file for each question, including all required HTML and JavaScript files.
- Push all final files to GitHub before submitting the link.
- Make sure the repository is accessible to the instructor.

Question 1 - Callback-Based Calculator and Bill

A shop needs reusable calculation functions. The calculation logic should be provided when the function is called.

- Create a function named calculate() that accepts two numbers and a callback. Pass a callback that adds the numbers and displays the result.
- Create a function named calculateBill() that accepts three product prices and a callback. Use prices 200, 300, and 500; the expected total is 1000.
- Create withdrawMoney(amount, callback). Assume a starting balance of 10,000. After approximately two seconds, call the callback to display the remaining balance.
- Make sure the callback runs after the delayed operation finishes.

Question 2 - Order Processing Sequence

An online food service must prepare an order, pack it, and then notify the customer. Each step takes approximately one second.

- Display a message when each step starts and when it finishes.
- Make sure packing starts only after preparation finishes, and notification happens only after packing finishes.
- First implement the workflow using nested callbacks to demonstrate how deeply nested asynchronous code can become.
- Write a second version of the workflow using a cleaner Promise-based approach or async/await.
- Add a short comment comparing the readability of the two versions.

Question 3 - Event Registration Promise

A training institute has an online registration service. Registration succeeds when a seat is available and fails when the class is full.

- Create a function named registerStudent() that returns a Promise and accepts a student name and whether a seat is available.
- If a seat is available, fulfill the Promise with a registration confirmation message.
- If no seat is available, reject the Promise with a useful error message.

- Handle the result and error using Promise chaining.
- Test both cases: one registration with a seat available and one with no seat available. Display the messages using console.log().

Question 4 - Load Student Records in Sequence

A school portal loads details for three students. Each student's data takes about two seconds to load, and each student should be shown before the next student's request begins.

- Create getStudentData(id) that returns a Promise and resolves with sample student details after a two-second delay.
- Use student IDs 1, 2, and 3.
- Display a loading message and then display each student's data when it is ready.
- Load Student 1, then Student 2, then Student 3, in that order, using async/await.
- Add error handling so a rejected request does not cause an unhandled error.
- Add a comment explaining why await pauses the current async function but does not freeze the whole JavaScript program.

Question 6 - Mini Project: Support Ticket Status Checker

A help-desk website lets users check the status of a support ticket. Create a small ticket checker with asynchronous behavior.

- Create sample ticket data containing a ticket ID, customer name, and status (for example, Open, In Progress, or Resolved).
- Create a function that simulates retrieving a ticket and returns a Promise that settles after approximately two seconds.
- Use async/await to retrieve and display the selected ticket.
- If the ticket ID does not exist, reject the operation and display a helpful message.
- Display a loading message during retrieval and handle errors gracefully.
- You may create an HTML input and button for the ticket ID, or use a predefined ticket ID in JavaScript.

Suggested File Structure

```
Assignment-10/
  Q1_Callbacks.html / Q1_Callbacks.js
  Q2_OrderProcessing.html / Q2_OrderProcessing.js
  Q3_Promises.html / Q3_Promises.js
  Q4_AsyncAwait.html / Q4_AsyncAwait.js
  Q6_TicketChecker.html / Q6_TicketChecker.js
```

Final Submission Checklist

- Separate files created for each question.
- Callbacks and delayed operations demonstrated.
- Sequential asynchronous steps implemented.
- Promise fulfillment and rejection handled.
- Promise chaining and async/await demonstrated.
- Error cases tested and handled.
- Code tested in the browser.
- All files pushed to GitHub and repository link ready.

Work through the sequence, handle failures, and keep your async code readable. Happy Coding!